

Introducción a la programación en Python

Pedro Corcuera

Dpto. Matemática Aplicada y
Ciencias de la Computación

Universidad de Cantabria

`corcuerp@unican.es`



Objetivos

- Programación orientada a objetos
- Estructuras de datos



Índice

- Programación orientada a objetos
- Búsqueda y ordenación
- Estructuras de datos: Pilas, Colas



Limitaciones de la programación estructurada

- La descomposición de programas en funciones no es suficiente para conseguir la reusabilidad de código.
- Es difícil entender, mantener y actualizar un programa que consiste de una gran colección de funciones.
- Con la programación orientada a objeto se supera las limitaciones de la programación estructurada mediante un estilo de programación en el que las tareas se resuelven mediante la colaboración de objetos.

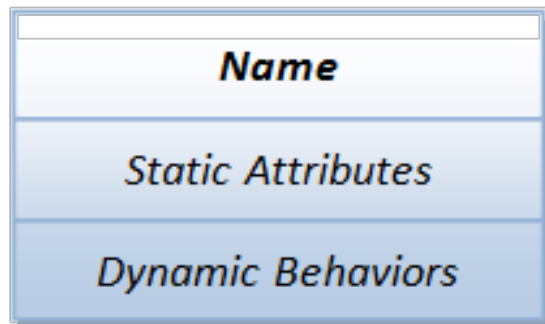


Programación orientada a objetos (OOP) en Python

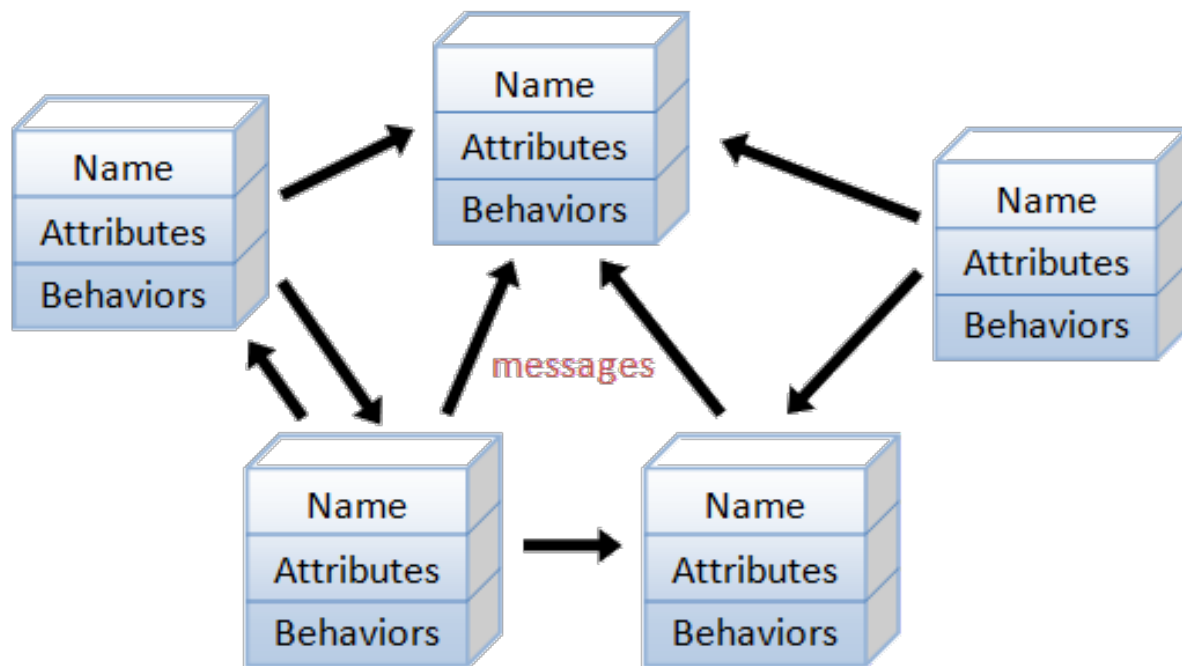
- Una clase es una plantilla de entidades del mismo tipo. Una instancia es una realización particular de una clase. Python soporta instancias de clases y objetos.
- Un objeto contiene atributos: atributos de datos (o variables) y comportamiento (llamados métodos). Para acceder un atributo se usa el operador punto, ejemplo: `nombre_instancia.nombre_atributo`
- Para crear una instancia de una clase se invoca el constructor: `nombre_instancia = nombre_clase(args)`



Clases – representación UML



A class is a 3-compartment box



An object-oriented program consists of many well-encapsulated objects and interacting with each other by sending messages



Objetos de clase vs Objetos de instancia

- Los objetos de clase sirven como factorías para generar objetos de instancia. Los objetos instanciados son objetos reales creados por una aplicación. Tienen su propio espacio de nombres.
- La instrucción `class` crea un objeto de clase con el nombre de clase. Dentro de la definición de la clase se puede crear variables de clase y métodos mediante `defs` que serán compartidos por todas las instancias



- ```
class class_name(superclass1, ...):
 """Class doc-string"""
 class_var1 = value1 # Class variables

 def __init__(self, arg1, ...):
 """Constructor"""
 self.instance_var1 = arg1 # inst var by assignment

 def __str__(self):
 """For printf() and str()"""

 def __repr__(self):
 """For repr() and interactive prompt"""

 def method_name(self, *args, **kwargs):
 """Method doc-string"""

```





# Constructores

- Un constructor es un método que inicializa las variables de instancia de un objeto
  - Se invoca automáticamente cuando se crea un objeto.

```
register = CashRegister()
```

Crea una instancia  
de CashRegister

- Python usa el nombre especial `__init__` para el constructor porque el propósito es inicializar una instancia de la clase.



## Constructor: Self

- El primer parámetro de un constructor debe ser `self`
- Cuando se invoca el constructor para crear un nuevo objeto, el parámetro `self` se asigna al objeto que esta siendo inicializado

```
def __init__(self):
 self._itemCount = 0
 self._totalPrice = 0
```

Referencia al  
objeto inicializado

```
register = CashRegister()
```

Crea una instancia  
de CashRegister

Referencia al objeto creado cuando  
el constructor termina



## Ejemplo – circle.py

| <b>circle</b>                 |   |
|-------------------------------|---|
| radius:float = 1.0            |   |
| <code>__init__()</code>       | • |
| <code>__str__():str</code>    | • |
| <code>__repr__():str</code>   | • |
| <code>get_area():float</code> |   |

Instance initializer

For `str()` and `print()`

For `repr()` and prompt



## Ejemplo – circle.py

- circle.py:

```
from math import pi
```

```
class Circle:
```

```
 """A Circle instance models a circle with a radius"""
```

```
 def __init__(self, radius=1.0):
```

```
 """Constructor with default radius of 1.0"""
```

```
 self.radius = radius # Create an inst var radius
```

```
 def __str__(self):
```

```
 """Return string, invoked by print() and str()"""
```

```
 return 'circle with radius of %.2f' % self.radius
```

```
 def __repr__(self):
```

```
 """Return string used to re-create this instance"""
```

```
 return 'Circle(radius=%f)' % self.radius
```

```
 def get_area(self):
```

```
 """Return the area of this Circle instance"""
```

```
 return self.radius * self.radius * pi
```



## Ejemplo – circle.py

- circle.py (cont.):

```
if run under Python interpreter, __name__ is '__main__'.
If imported into another module, __name__ is 'Circle'
if __name__ == '__main__':
 c1 = Circle(2.1) # Construct an instance
 print(c1) # Invoke __str__()
 print(c1.get_area())
 c2 = Circle() # Default radius
 print(c2)
 print(c2.get_area()) # Invoke member method
 c2.color = 'red' # Create new attribute via assignment
 print(c2.color)
 #print(c1.color) # Error - c1 has no attribute color
 # Test doc-strings
 print(__doc__) # This module
 print(Circle.__doc__) # Circle class
 print(Circle.get_area.__doc__) # get_area() method
 print(isinstance(c1, Circle)) # True
```



# Construcción de clase

---

- Para construir una instancia de una instancia se realiza a través del constructor `nombre_clase(...)`

```
c1 = Circle(1.2)
c2 = Circle() # radius default
```

- Python crea un objeto `Circle`, luego invoca el método `__init__(self, radius)` con `self` asignado a la nueva instancia
  - `__init__()` es un inicializador para crear variables de instancia
  - `__init__()` nunca devuelve un valor
  - `__init__()` es opcional y se puede omitir si no hay variables de instancia



# Métodos de instancia

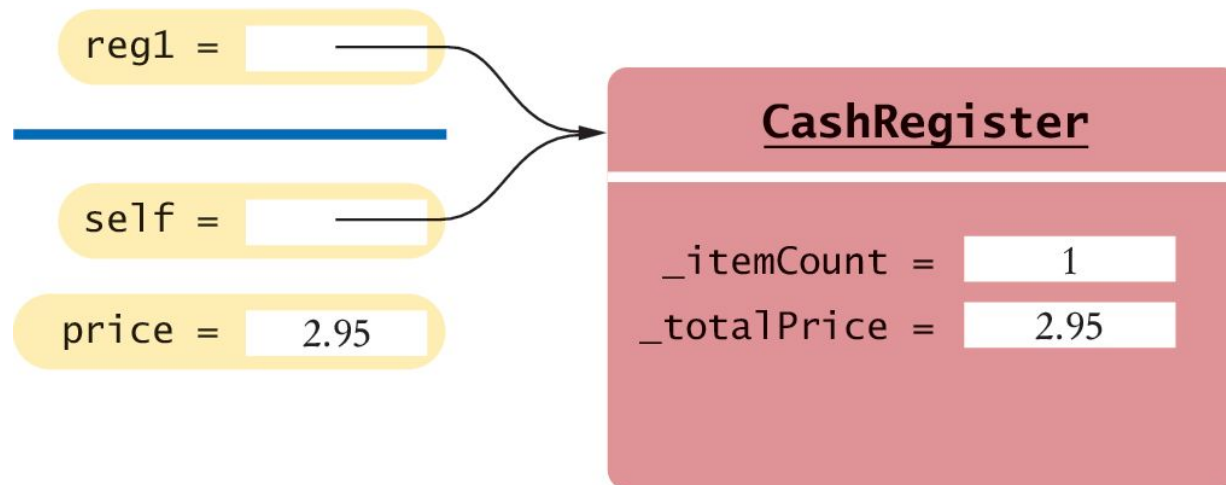
---

- Para invocar un método se usa el operador punto, en la forma `nombre_obj.nombre metodo ( )`. Python diferencia entre instancias de objetos y objetos de clase:
  - Para objetos de clase: se puede invocar  
`class_name.method_name(instance_name, ...)`
  - Para instancia de objetos:  
`instance_name.method_name(...)`  
donde `instance_name` es pasado en el método como el argumento 'self'



# La referencia self

- Cada método tiene una referencia al objeto sobre el cual el método fue invocado, almacenado en el la variable parámetro `self`.



```
def addItem(self, price):
 self.itemCount = self.itemCount + 1
 self.totalPrice = self.totalPrice + price
```





# Clase Point y sobrecarga de operadores

- Ejemplo point.py que modela un punto 2D con coordenadas x e y. Se sobrecarga los operadores + y \*.

| Point                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------|
| x:int = 0<br>y:int = 0                                                                                                                       |
| <code>__init__()</code><br><code>__str__():str</code><br><code>__repr__():str</code><br><code>__add__()</code> •<br><code>__mul__()</code> • |

Override to overload '+' operator

Override to overload '\*' operator



# Métodos especiales (mágicos)

| Magic Method                                                                                                                                                                                                                                                             | Invoked Via          | Invocation Syntax                                                                                                                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>__lt__(self, right)</code><br><code>__gt__(self, right)</code><br><code>__le__(self, right)</code><br><code>__ge__(self, right)</code><br><code>__eq__(self, right)</code><br><code>__ne__(self, right)</code>                                                     | Comparison Operators | <code>self &lt; right</code><br><code>self &gt; right</code><br><code>self &lt;= right</code><br><code>self &gt;= right</code><br><code>self == right</code><br><code>self != right</code>                |
| <code>__add__(self, right)</code><br><code>__sub__(self, right)</code><br><code>__mul__(self, right)</code><br><code>__truediv__(self, right)</code><br><code>__floordiv__(self, right)</code><br><code>__mod__(self, right)</code><br><code>__pow__(self, right)</code> | Arithmetic Operators | <code>self + right</code><br><code>self - right</code><br><code>self * right</code><br><code>self / right</code><br><code>self // right</code><br><code>self % right</code><br><code>self ** right</code> |



# Métodos especiales (mágicos)

| Magic Method                                                                                                                                                                                                                                                  | Invoked Via                    | Invocation Syntax                                                                                                                                                                                   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>__and__(self, right)</code><br><code>__or__(self, right)</code><br><code>__xor__(self, right)</code><br><code>__invert__(self)</code><br><code>__lshift__(self, n)</code><br><code>__rshift__(self, n)</code>                                           | Bitwise Operators              | <code>self &amp; right</code><br><code>self   right</code><br><code>self ^ right</code><br><code>~self</code><br><code>self &lt;&lt; n</code><br><code>self &gt;&gt; n</code>                       |
| <code>__str__(self)</code><br><code>__repr__(self)</code><br><code>__sizeof__(self)</code>                                                                                                                                                                    | Function call                  | <code>str(self)</code> , <code>print(self)</code><br><code>repr(self)</code><br><code>sizeof(self)</code>                                                                                           |
| <code>__len__(self)</code><br><code>__contains__(self, item)</code><br><code>__iter__(self)</code><br><code>__next__(self)</code><br><code>__getitem__(self, key)</code><br><code>__setitem__(self, key, value)</code><br><code>__delitem__(self, key)</code> | Sequence Operators & Functions | <code>len(self)</code><br><code>item in self</code><br><code>iter(self)</code><br><code>next(self)</code><br><code>self[key]</code><br><code>self[key] = value</code><br><code>del self[key]</code> |



# Métodos especiales (mágicos)

| Magic Method                                                                                                                                          | Invoked Via                                 | Invocation Syntax                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <code>__int__(self)</code><br><code>__float__(self)</code><br><code>__bool__(self)</code><br><code>__oct__(self)</code><br><code>__hex__(self)</code> | Type Conversion Function call               | <code>int(self)</code><br><code>float(self)</code><br><code>bool(self)</code><br><code>oct(self)</code><br><code>hex(self)</code> |
| <code>__init__(self, *args)</code><br><code>__new__(cls, *args)</code>                                                                                | Constructor                                 | <code>x = ClassName(*args)</code>                                                                                                 |
| <code>__del__(self)</code>                                                                                                                            | Operator <code>del</code>                   | <code>del x</code>                                                                                                                |
| <code>__index__(self)</code>                                                                                                                          | Convert this object to an index             | <code>x[self]</code>                                                                                                              |
| <code>__radd__(self, left)</code><br><code>__rsub__(self, left)</code><br>...                                                                         | RHS (Reflected) addition, subtraction, etc. | <code>left + self</code><br><code>left - self</code><br>...                                                                       |
| <code>__iadd__(self, right)</code><br><code>__isub__(self, right)</code><br>...                                                                       | In-place addition, subtraction, etc         | <code>self += right</code><br><code>self -= right</code><br>...                                                                   |



# Métodos especiales (mágicos)

| Magic Method                                                                                                                | Invoked Via                           | Invocation Syntax                                                                                           |
|-----------------------------------------------------------------------------------------------------------------------------|---------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>__pos__(self)</code><br><code>__neg__(self)</code>                                                                    | Unary Positive and Negative operators | <code>+self</code><br><code>-self</code>                                                                    |
| <code>__round__(self)</code><br><code>__floor__(self)</code><br><code>__ceil__(self)</code><br><code>__trunc__(self)</code> | Function Call                         | <code>round(self)</code><br><code>floor(self)</code><br><code>ceil(self)</code><br><code>trunc(self)</code> |
| <code>__getattr__(self, name)</code><br><code>__setattr__(self, name, value)</code><br><code>__delattr__(self, name)</code> | Object's attributes                   | <code>self.name</code><br><code>self.name = value</code><br><code>del self.name</code>                      |
| <code>__call__(self, *args, **kwargs)</code>                                                                                | Callable Object                       | <code>obj(*args, **kwargs);</code>                                                                          |
| <code>__enter__(self)</code> , <code>__exit__(self)</code>                                                                  | Context Manager with-statement        |                                                                                                             |



# Clase Point y sobrecarga de operadores

---

- point.py modela un punto 2D con coordenadas x e y.

Se sobrecarga los operadores + y \*:

```
""" point.py: point module defines the Point class"""
class Point:
 """A Point models a 2D point x and y coordinates"""
 def __init__(self, x=0, y=0):
 """Constructor x and y with default of (0,0)"""
 self.x = x
 self.y = y
 def __str__(self):
 """Return a descriptive string for this instance"""
 return '("%.2f, %.2f)' % (self.x, self.y)
 def __add__(self, right):
 """Override the '+' operator"""
 p = Point(self.x + right.x, self.y + right.y)
 return p
```



# Clase Point y sobrecarga de operadores

---

```
def __mul__(self, factor):
 """Override the '*' operator"""
 self.x *= factor
 self.y *= factor
 return self

Test
if __name__ == '__main__':
 p1 = Point()
 print(p1) # (0.00, 0.00)
 p1.x = 5
 p1.y = 6
 print(p1) # (5.00, 6.00)
 p2 = Point(3, 4)
 print(p2) # (3.00, 4.00)
 print(p1 + p2) # (8.00, 10.00) Same as p1.__add__(p2)
 print(p1) # (5.00, 6.00) No change
 print(p2 * 3) # (9.00, 12.00) Same as p1.__mul__(p2)
 print(p2) # (9.00, 12.00) Changed
```

---



# Variables de clase

- Son valores que pertenecen a la clase y no al objeto de la clase.
- Las variables de clase se llaman también “variables estáticas”
- Las variables de clase se declaran al mismo nivel que los métodos.
- Las variables de clase siempre deben ser privadas para asegurar que métodos de otras clases no cambian sus valores. Las *constantes* de clase pueden ser públicas





## Variables de clase - Ejemplo

---

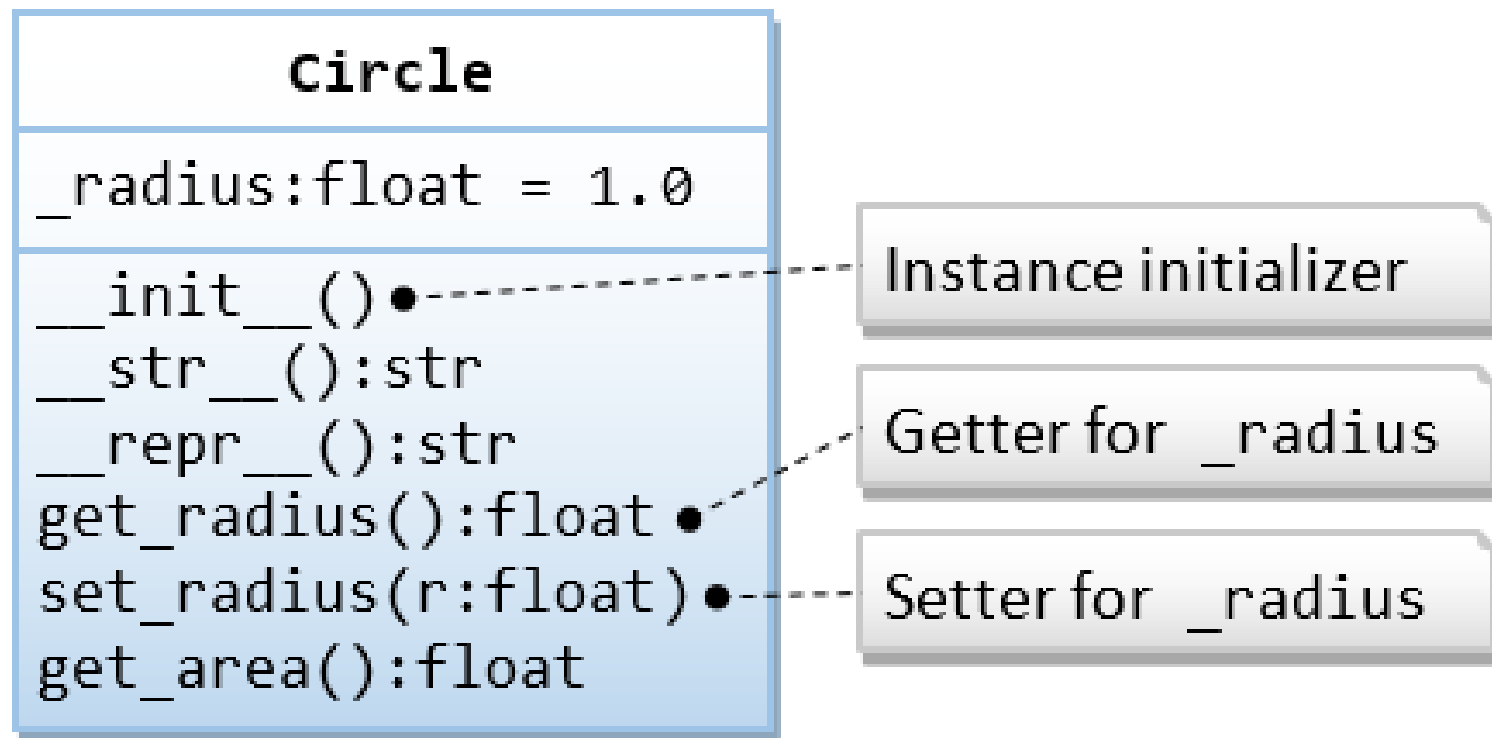
- Se desea asignar números de cuenta de banco secuencialmente empezando por 1001.

```
class BankAccount :
 _lastAssignedNumber = 1000 # A class variable
 OVERDRAFT_FEE = 29.95 # Class constant
 def __init__(self) :
 self._balance = 0
 BankAccount._lastAssignedNumber =
 BankAccount._lastAssignedNumber + 1
 self._accountNumber =
 BankAccount._lastAssignedNumber
```



## Métodos de acceso (get) y asignación (set)

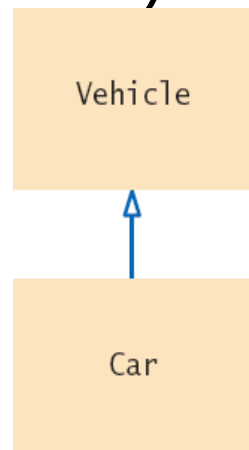
- Mejora de la clase Circle con métodos get y set para acceder las variables de instancia (circle1.py)





# Jerarquía de herencia

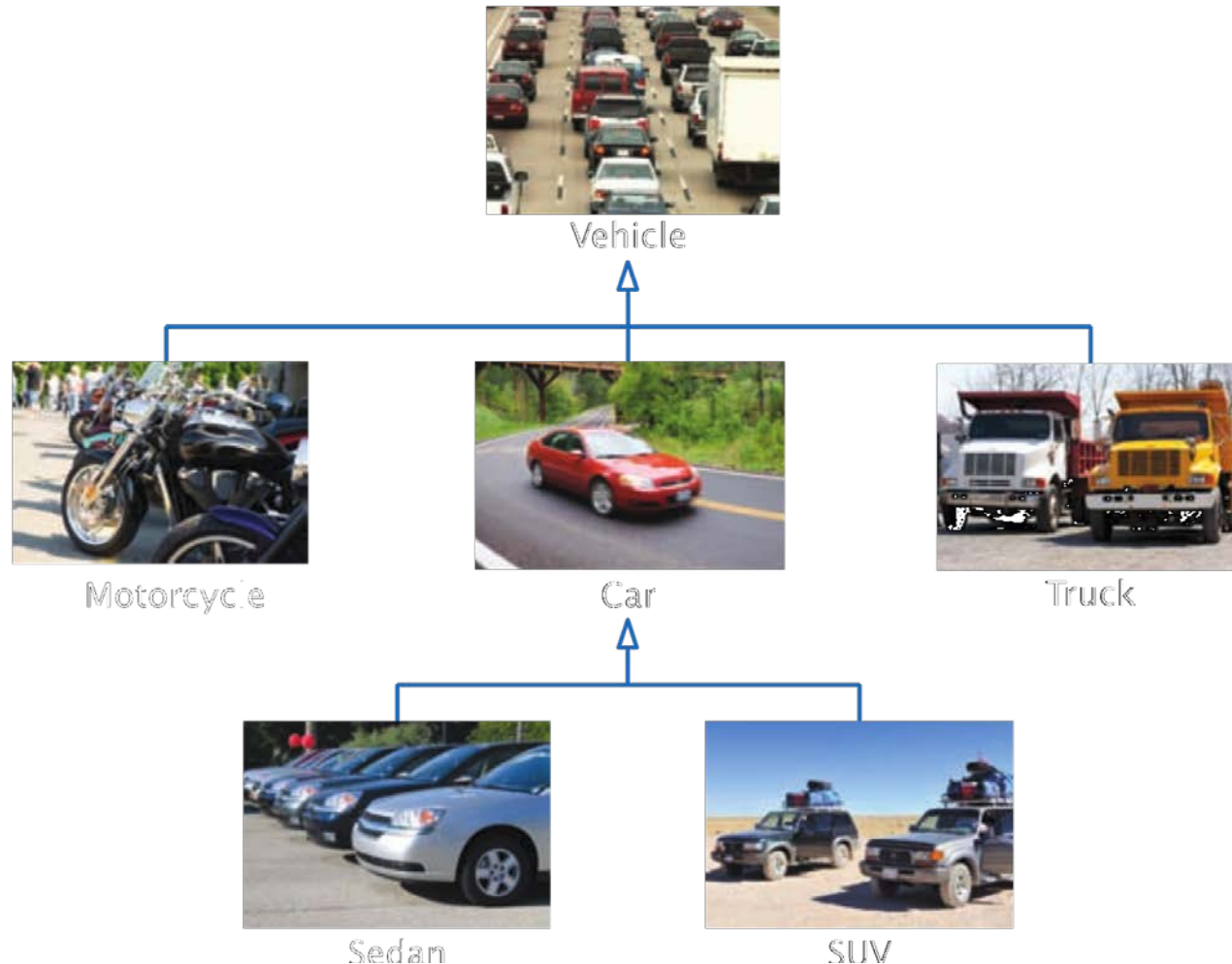
- En programación orientada a objetos, la herencia es una relación entre:
  - Una superclase: clase más general
  - Una subclase: clase más especializada
- La subclase 'hereda' los datos (variables) y comportamiento (métodos) de la superclase.





# Jerarquía de la clase Vehicle

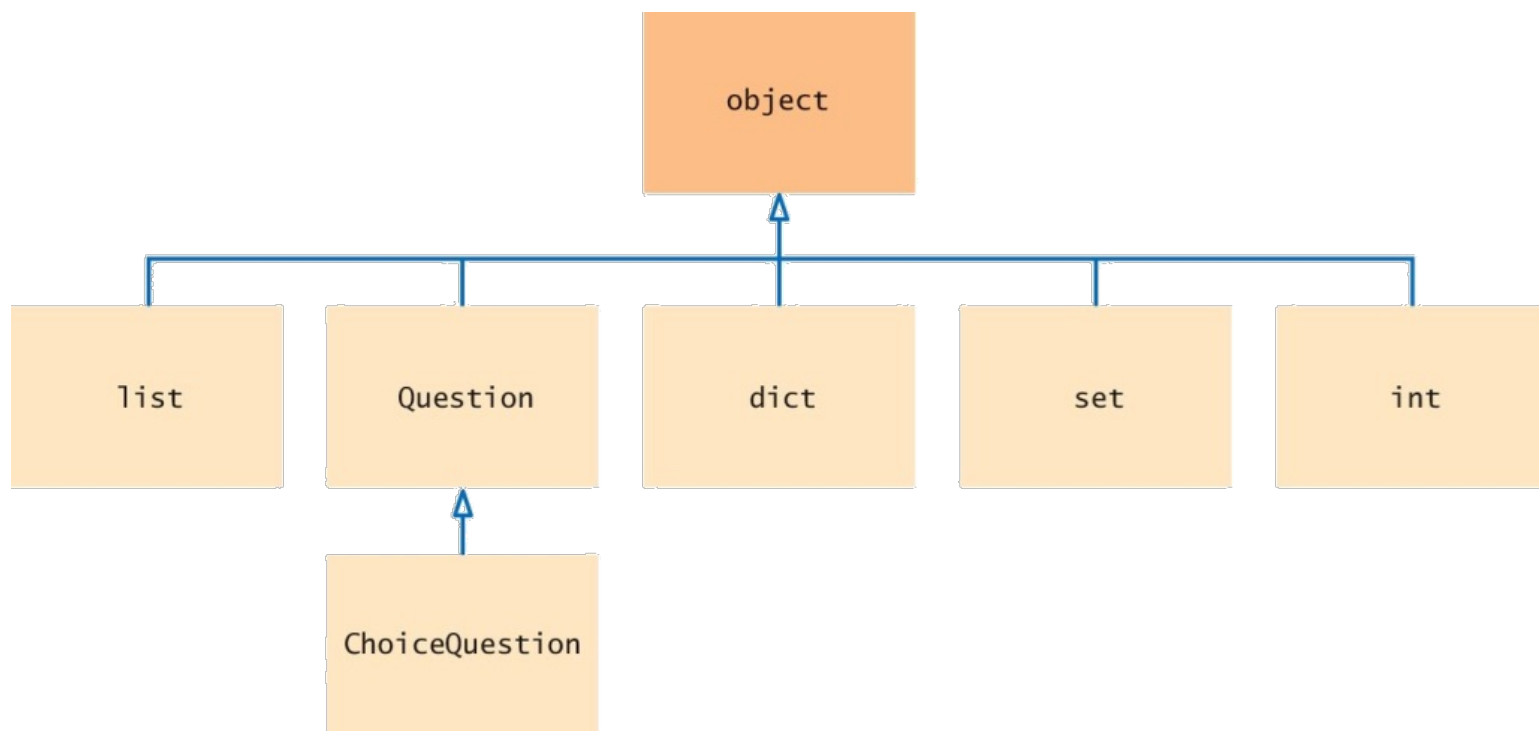
- General
- Especializado
- Más específico





# La superclase object

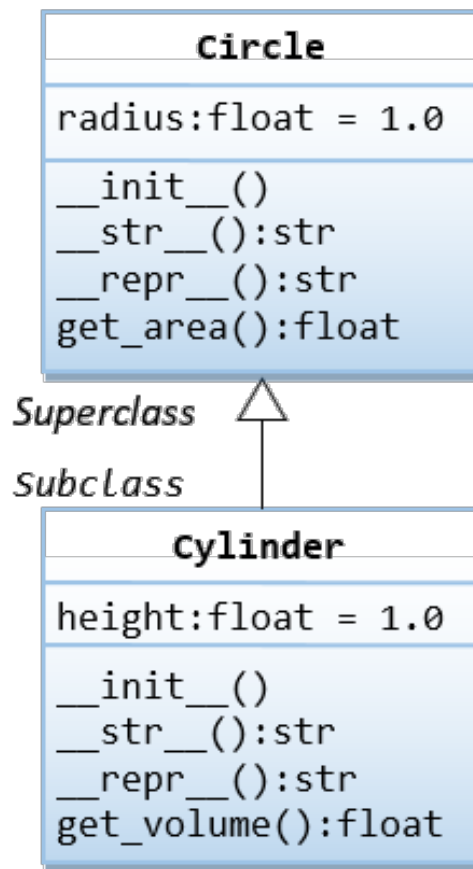
- En Python, cada clase que es declarada sin una superclase explícita automáticamente extiende la clase object.





# Herencia y polimorfismo

- Se puede definir una clase Cylinder como una subclase de Circle.



cylinder.py

Hereda atributo radius y método get\_area()



# Herencia

- cylinder.py un cilindro se puede derivar de un circulo

```
"""cylinder.py: which defines the Cylinder class"""
from circle import Circle # Using the Circle class
class Cylinder(Circle):
 """The Cylinder class is a subclass of Circle"""
 def __init__(self, radius = 1.0, height = 1.0):
 """Constructor"""
 super().__init__(radius) # Invoke superclass
 self.height = height
 def __str__(self):
 """Self Description for print()"""
 return 'Cylinder(radius=%.2f,height=%.2f)' %
(self.radius, self.height)
 def get_volume(self):
 """Return the volume of the cylinder"""
 return self.get_area() * self.height
```



# Herencia

- cylinder.py (cont.)

```
if __name__ == '__main__':
 cy1 = Cylinder(1.1, 2.2)
 print(cy1)
 print(cy1.get_area()) # inherited superclass' method
 print(cy1.get_volume()) # Invoke its method
 cy2 = Cylinder() # Default radius and height
 print(cy2)
 print(cy2.get_area())
 print(cy2.get_volume())
 print(dir(cy1))
 print(Cylinder.get_area)
 print(Circle.get_area)
 c1 = Circle(3.3)
 print(c1) # Output: circle with radius of 3.30
 print(issubclass(Cylinder, Circle)) # True
 print(issubclass(Circle, Cylinder)) # False
 print(isinstance(cy1, Cylinder)) # True
```

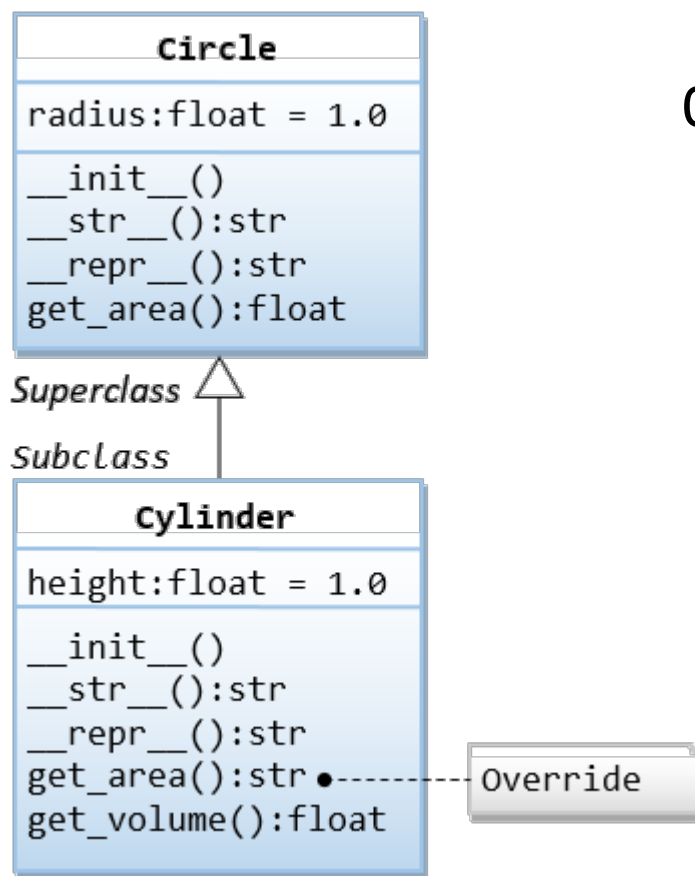




# Sobreescritura (overriding) de métodos

- Se puede sobreescribir el método `get_area()` para calcular la superficie del cilindro. También `__str__()`

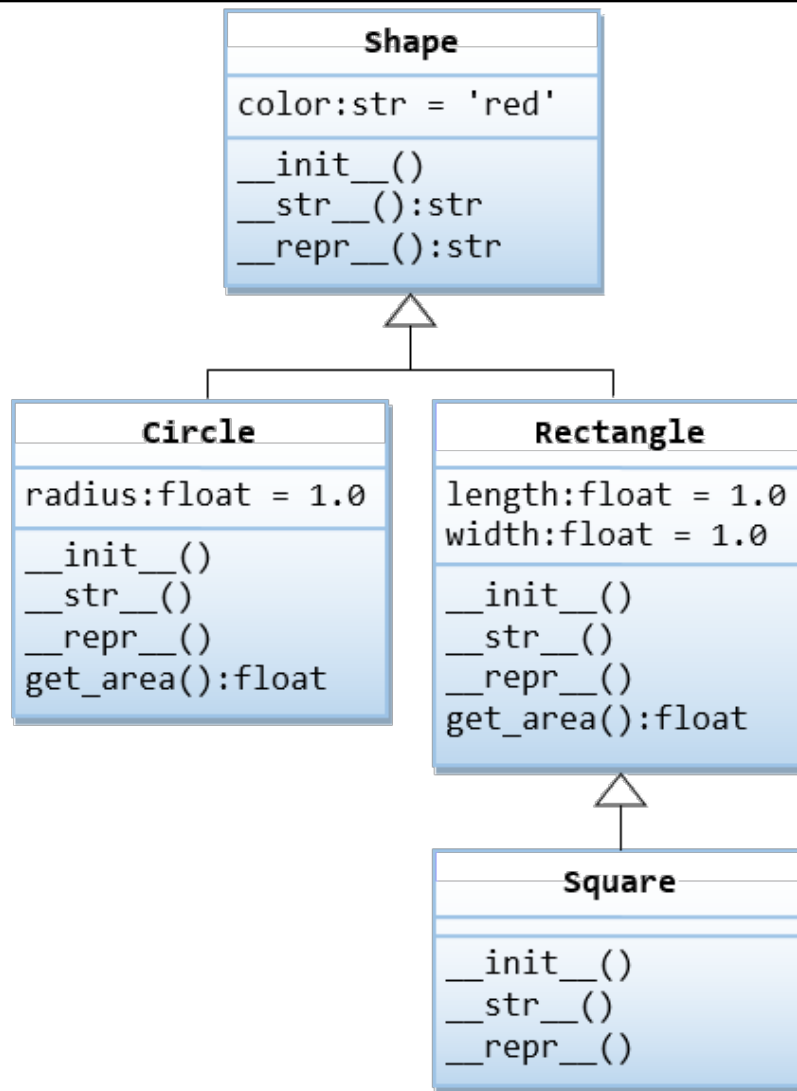
cylinder1.py





# Ejemplo shape y subclases

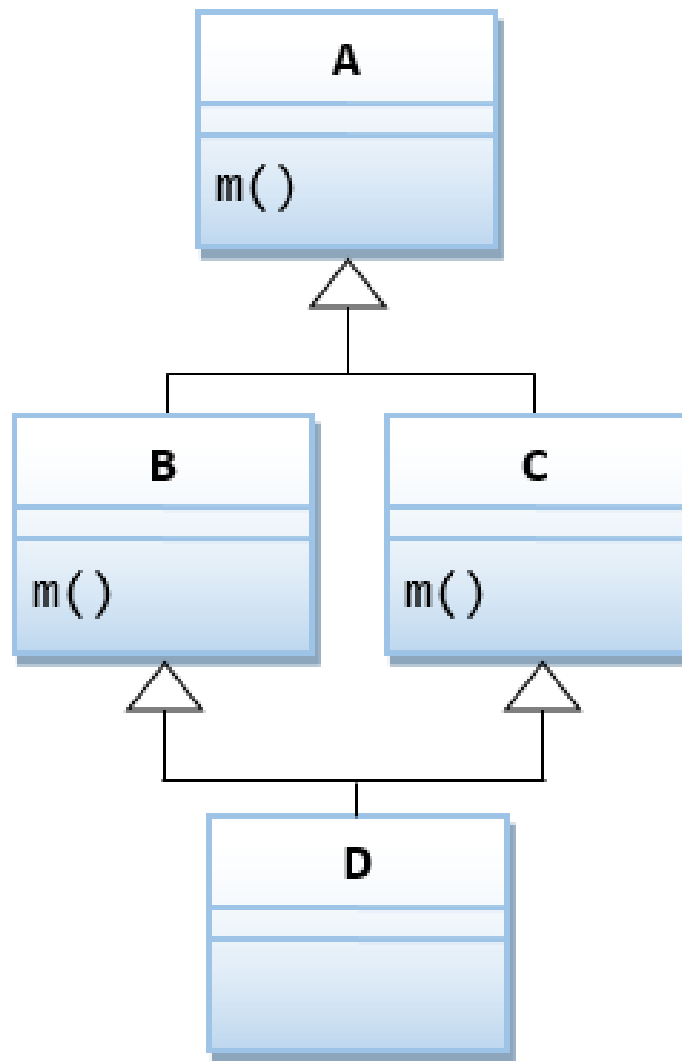
- sh.py





# Herencia múltiple

- minh.py
- minh1.py
- minh2.py
- minh3.py





# Tipo de dato definido por el usuario – Clase Charge

- Se define un tipo Charge para partículas cargadas.
- Se usa la ley de Coulomb para el cálculo del potencial de un punto debido a una carga  $V=kq/r$ , donde  $q$  es el valor de la carga,  $r$  es la distancia del punto a la carga y  $k=8.99 \times 10^9 \text{ N m}^2/\text{C}^2$ .

Constructor

*operation*

*description*

→ `Charge(x0, y0, q0)`

*a new charge centered at (x0, y0) with charge value q0*

`c.potentialAt(x, y)`

*electric potential of charge c at point (x, y)*

`str(c)`

*'q0 at (x0, y0)' (string representation of charge c)*

*API for our user-defined Charge data type*



## Convenciones sobre ficheros

---

- El código que define el tipo de dato definido por el usuario Charge se coloca en un fichero del mismo nombre (sin mayúscula) charge.py
- Un programa cliente que usa el tipo de dato Charge se pone en el cabecero del programa:

```
from charge import Charge
```



# Creación de objetos, llamada de métodos y representación de String

---

```
#-----
chargeclient.py
#-----
import sys
from charge import Charge
Acepta floats x e y como argumentos en la línea de comandos. Crea dos objetos
Charge con posición y carga. Imprime el potencial en (x, y) en la salida estandard
x = float(sys.argv[1])
y = float(sys.argv[2])
c1 = Charge(.51, .63, 21.3)
c2 = Charge(.13, .94, 81.9)
v1 = c1.potentialAt(x, y)
v2 = c2.potentialAt(x, y)
print('potential at (%.2f, %.2f) due to\n', x, y)
print(' ' + str(c1) + ' and')
print(' ' + str(c2))
print('is %.2e\n', v1+v2)
#-----
python chargeclient.py .2 .5
potential at (0.20, 0.50) due to
21.3 at (0.51, 0.63) and
81.9 at (0.13, 0.94)
is 2.22e+12
```



# Elementos básicos de un tipo de dato

- API

| <i>operation</i>                                 | <i>description</i>                                                                         |
|--------------------------------------------------|--------------------------------------------------------------------------------------------|
| <code>Charge(x0, y0, q0)</code>                  | <i>a new charge centered at <math>(x_0, y_0)</math> with charge value <math>q_0</math></i> |
| <code>c.potentialAt(x, y)</code>                 | <i>electric potential of charge <math>c</math> at point <math>(x, y)</math></i>            |
| <code>str(c)</code>                              | <i>'q0 at (x0, y0)' (string representation of charge <math>c</math>)</i>                   |
| <i>API for our user-defined Charge data type</i> |                                                                                            |

- Clase. Fichero charge.py. Palabra reservada class
- Constructor. Método especial `__init__()`, self
- Variable de instancia. `_nombrevar`
- Métodos. Variable de instancia self
- Funciones intrínsecas. `__str__`
- Privacidad



# Implementación de Charge

```
import sys
import math

En charge.py

class Charge:
 def __init__(self, x0, y0, q0):
 self._rx = x0 # x value of the query point
 self._ry = y0 # y value of the query point
 self._q = q0 # Charge

 def potentialAt(self, x, y):
 COULOMB = 8.99e09
 dx = x - self._rx
 dy = y - self._ry
 r = math.sqrt(dx*dx + dy*dy)
 if r == 0.0: # Avoid division by 0
 if self._q >= 0.0:
 return float('inf')
 else:
 return float('-inf')
 return COULOMB * self._q / r

 def __str__(self):
 result = str(self._q) + ' at (' + str(self._rx) + ', ' + str(self._ry) + ')'
 return result

if __name__ == '__main__': main()
```





# Clase Complex

- Métodos especiales. En Python la expresión  $a * b$  se reemplaza con la llamada del método `a.__mul__(b)`

| <i>client operation</i>                         | <i>special method</i>               | <i>description</i>                                     |
|-------------------------------------------------|-------------------------------------|--------------------------------------------------------|
| <code>Complex(x, y)</code>                      | <code>__init__(self, re, im)</code> | <i>new Complex object with value <math>x+yi</math></i> |
| <code>a.re()</code>                             |                                     | <i>real part of a</i>                                  |
| <code>a.im()</code>                             |                                     | <i>imaginary part of a</i>                             |
| <code>a + b</code>                              | <code>__add__(self, other)</code>   | <i>sum of a and b</i>                                  |
| <code>a * b</code>                              | <code>__mul__(self, other)</code>   | <i>product of a and b</i>                              |
| <code>abs(a)</code>                             | <code>__abs__(self)</code>          | <i>magnitude of a</i>                                  |
| <code>str(a)</code>                             | <code>__str__(self)</code>          | <i>'x + yi' (string representation of a)</i>           |
| <i>API for a user-defined Complex data type</i> |                                     |                                                        |



# Sobrecarga de operadores

| <i>client operation</i> | <i>special method</i>                  | <i>description</i>                    |
|-------------------------|----------------------------------------|---------------------------------------|
| <code>x + y</code>      | <code>__add__(self, other)</code>      | <i>sum of x and y</i>                 |
| <code>x - y</code>      | <code>__sub__(self, other)</code>      | <i>difference of x and y</i>          |
| <code>x * y</code>      | <code>__mul__(self, other)</code>      | <i>product of x and y</i>             |
| <code>x ** y</code>     | <code>__pow__(self, other)</code>      | <i>x to the yth power</i>             |
| <code>x / y</code>      | <code>__truediv__(self, other)</code>  | <i>quotient of x and y</i>            |
| <code>x // y</code>     | <code>__floordiv__(self, other)</code> | <i>floored quotient of x and y</i>    |
| <code>x % y</code>      | <code>__mod__(self, other)</code>      | <i>remainder when dividing x by y</i> |
| <code>+x</code>         | <code>__pos__(self)</code>             | <i>x</i>                              |
| <code>-x</code>         | <code>__neg__(self)</code>             | <i>arithmetic negation of x</i>       |

*Special methods for arithmetic operators*



# Algoritmos de búsqueda

---

- Secuencial  $O(n)$
- Búsqueda binaria  $O(\log n)$



# Algoritmos de ordenación

---

- Burbuja
- Inserción
- Selección
- Mezcla
- Quicksort



# Estructuras de datos: Pilas y colas

---

- Pilas. Colección basada en política LIFO
- Colas. Colección basada en política FIFO
- Árboles binarios de búsqueda
- Tabla de símbolos



# GUI con tkinter

---

- El paquete tkinter sirve para generar interfaces gráficas de usuario.
- [Tkinter documentation.](#)